LAB SHEET-9

Design and Development of a Full-Stack "Student Record Management" Web Application
To design, develop, and test a full-stack CRUD (Create, Read, Update, Delete) web application that integrates a frontend UI, a backend REST API, and a database, demonstrating the complete request-response cycle of a full-stack system.

server.js

```
const express=require("express");
const mongoose=require("mongoose");
const cors=require("cors");
const app=express();
app.use(cors());
app.use(express.json());
mongoose.connect("mongodb://127.0.0.1:27017/studentDB");
const Student=mongoose.model("Student",new
mongoose.Schema({name:String,rollNo:{type:String,required:true},course:String,marks:{type:Number,min:0,max:100}}));
app.post("/students",async(req,res)=>res.status(201).json(await Student.create(req.body)));
app.get("/students",async(req,res)=>res.json(await Student.find()));
app.get("/students/:id",async(req,res)=>res.json(await Student.findById(req.params.id)));
app.put("/students/:id",async(req,res)=>res.json(await
Student.findByIdAndUpdate(req.params.id,req.body,{new:true,runValidators:true})));
app.delete("/students/:id",async(req,res)=>{await Student.findByIdAndDelete(req.params.id);res.json({message:"Student
deleted"});});
app.listen(5000,()=>console.log("Server running on port 5000"));
```

index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Student Management</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h1>Student Record Management</h1>
<form id="form">
<input id="name" placeholder="Name" required>
<input id="rollNo" placeholder="Roll No." required>
<input id="course" placeholder="Course" required>
<input id="marks" type="number" min="0" max="100" placeholder="Marks" required>
<button>Add Student</button>
</form>
<h2>Student Records</h2>
<table>
<thead><tr><th>Name</th><th>Roll No.</th><th>Course</th><th>Marks</th><th>Actions</th></tr></thead>
<tbody id="table"></tbody>
</table>
</div>
<script src="script.js"></script>
</body>
</html>
```

script.js

```
const API="http://localhost:5000/students";
const form=document.getElementById("form"),table=document.getElementById("table");
let editId=null;
async function load(){
const data=await(await fetch(API)).json();
table.innerHTML=data.map(s=>`<tr><td>${s.name}</td><td>${s.rollNo}</td><td>${s.course}</td><td>${s.marks}</td><t
d><button onclick="edit('${s._id}')">Edit</button><button class="delete"
onclick="del('${s._id}')">Delete</button></td></tr>`).join("");
}
```

```
form.onsubmit=async e=>{
e.preventDefault();
const data={name:name.value.trim(),rollNo:rollNo.value.trim(),course:course.value.trim(),marks:Number(marks.value)};
if(!data.rollNo||data.marks<0||data.marks>100)return alert("Enter valid Roll No. and marks");
await fetch(editId?`${API}/${editId}`:API,{method:editId?"PUT":"POST",headers:{"Content-
Type":"application/json"},body:JSON.stringify(data)});
editId=null;form.reset();load();
};
async function edit(id){
const s=await(await fetch(`${API}/${id}`)).json();
name.value=s.name;rollNo.value=s.rollNo;course.value=s.course;marks.value=s.marks;editId=id;
}
async function del(id){
if(confirm("Delete this student?")){await fetch(`${API}/${id}`,{method:"DELETE"});load();}
}
load();
```

style.css

```
body{font-family:Arial,sans-serif;background:#E6E6FA;margin:0;padding:30px}
.container{max-width:900px;margin:auto;background:#FFF0F5;padding:25px;border-radius:10px}
h1,h2{text-align:center}
form{display:grid;gap:10px;margin-bottom:30px}
input{padding:10px;border:1px solid rgb(194,152,166);border-radius:5px}
button{padding:10px;border:none;border-radius:5px;cursor:pointer}
table{width:100%;border-collapse:collapse}
th,td{padding:12px;border:1px solid #ddd;text-align:center}
th{background:rgb(161,134,142)}
.edit{background:#ddd;margin-right:5px}
.delete{background:#ddd}
```

student.js

```
const mongoose=require("mongoose");
const studentSchema=new mongoose.Schema({
name:{type:String,required:true},
rollNo:{type:String,required:true,unique:true},
course:{type:String,required:true},
marks:{type:Number,required:true,min:0,max:100}
});
module.exports=mongoose.model("Student",studentSchema);
```

Login Screen of Admin

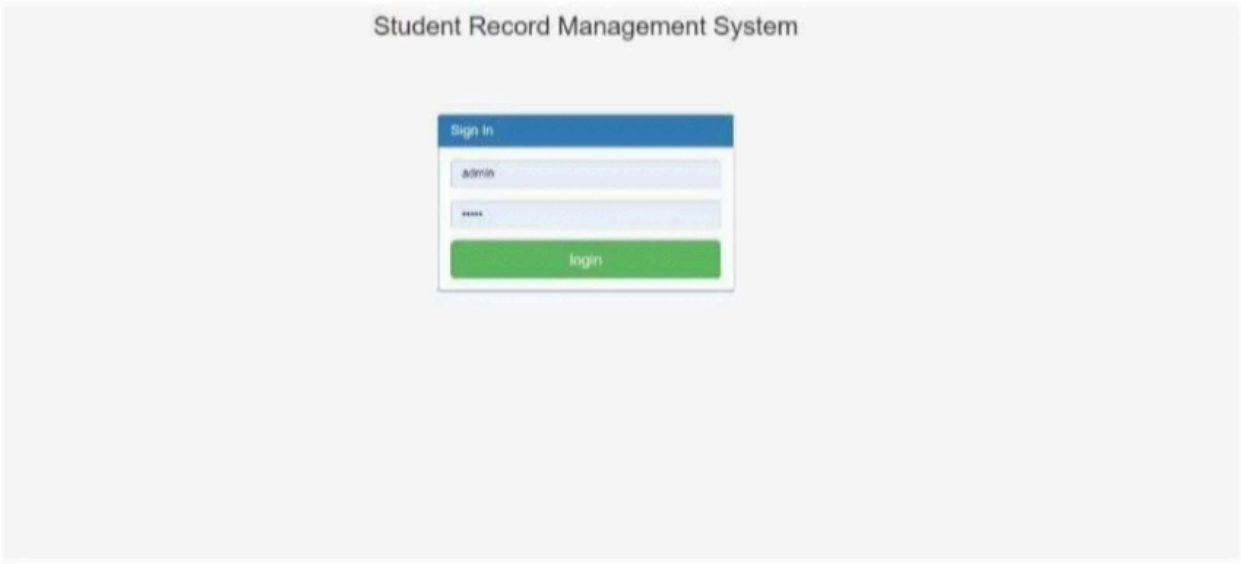


Fig – Login Screen of Admin

Dashboard

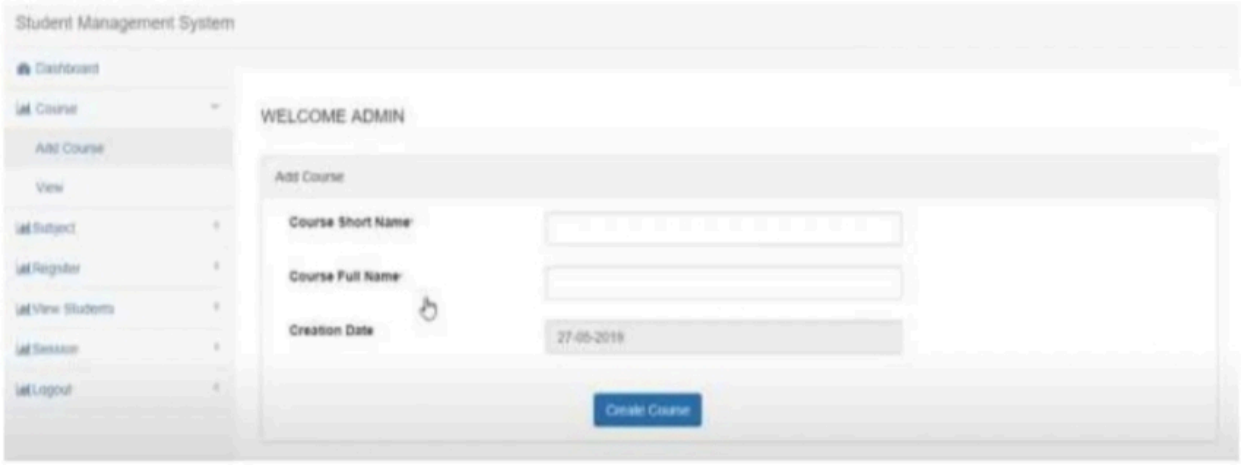
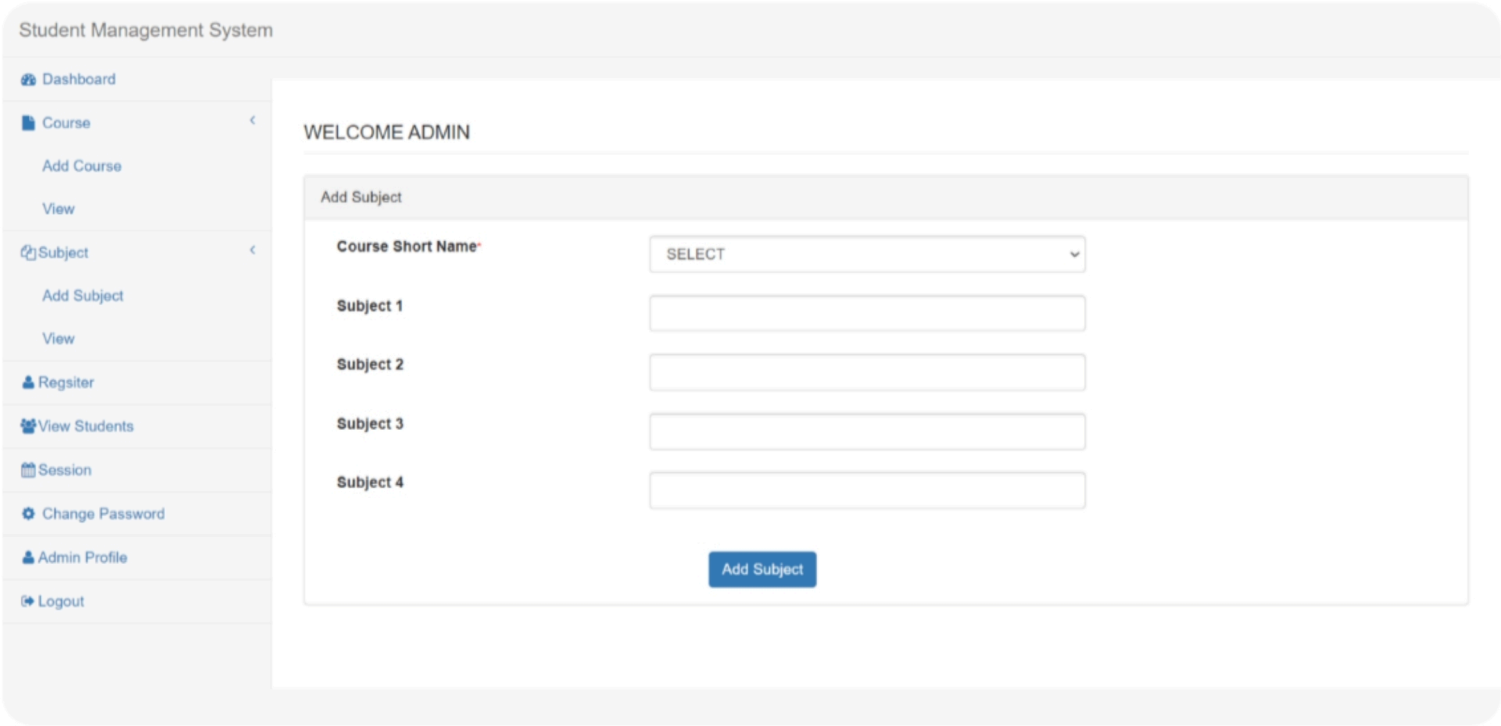


Fig – Dashboard


Student Management System
Dashboard
Course
Add Course
View
Subject
Add Subject
View
Regsiter
View Students
Session
Change Password
Admin Profile
Logout
WELCOME ADMIN
Add Subject
Course Short Name*
SELECT
Subject 1
Subject 2
Subject 3
Subject 4
Add Subject